Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Chapter 5

Design and analysis of Backtracking Algorithms

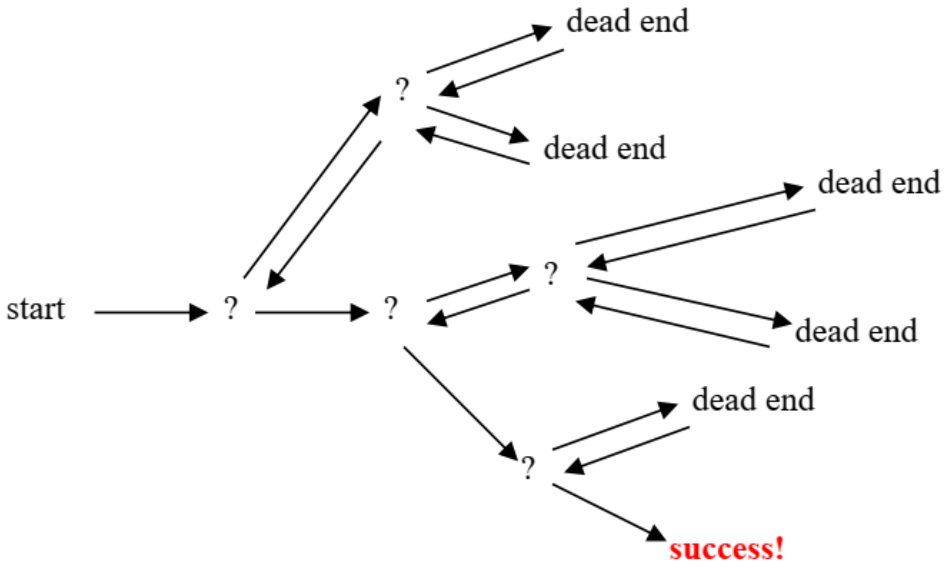
Dr Rahaf Ghazal

2025-2026

- الخوارزميات العودية (Recursive Algorithms)
- الخوارزميات التتبع الخلفي - التراجعية (Backtracking algorithms)
- خوارزميات البرمجة الديناميكية (Dynamic Programming Algorithms)
- خوارزميات التفريق ثم الحل (Divide and Conquer Algorithms)
- الخوارزميات الجشعة والأنانية (Greedy Algorithms)

Introduction

- **Backtracking algorithms** are like problem-solving strategies that help explore **different options** to find the best solution. They work by trying out different paths and if one doesn't work**(lead to a dead end),** they backtrack and try another until they find the right one. It's like solving a puzzle by testing different pieces until they fit together perfectly.
- It is commonly used in situations where you need to explore multiple possibilities to solve a problem, like *searching for a path in a maze* or *solving puzzles like Sudoku.*



الخوارزميات التراجعية هي خوارزميات عودية تعتمد طريقة "التجريب و الخطأ" (Try-and-Error) في حل المسائل

When to Use a Backtracking Algorithm?

Backtracking algorithms are best in following states:

- ***Constraint satisfaction problems:*** When you need to build a solution step by step and must satisfy certain conditions
- ***Multiple solutions:*** When you need to explore all possible valid solutions, not just one.
- ***Combinatorial problems:*** When you must generate all valid combinations or subsets under constraints.

Key Real-World Applications:

- ***Puzzles and Games:*** Sudoku, N-Queens, and chess AI.
- ***Pathfinding :*** Finding paths in mazes or routing for robotics, where the algorithm explores possible routes and backtracks when encountering obstacles or dead ends.
- ***Constraint Satisfaction Problems:*** Scheduling tasks and map coloring, where specific rules must be satisfied.
- ***Combinatorial Optimization:*** Solving problems like the Knapsack problem to find optimal solutions under constraints.

When Not to Use Backtracking

- Greedy or DP fits better: If the problem can be solved directly using a greedy strategy or dynamic programming, backtracking is overkill.
- No pruning possible: If all branches must be explored anyway (no constraints to cut early), brute force or iterative methods may be simpler.
- Large input size: Backtracking can be exponential in time. For very large inputs without strong pruning opportunities, it becomes impractical.
- Single optimal solution: If the task only needs one best solution with clear optimization criteria, algorithms like DP, greedy, or graph search may be faster.

How Does a Backtracking Algorithm Work?

The following is a general outline of how a backtracking algorithm works:

- Choose an initial solution.
- Explore all possible extensions of the current solution.
- If an extension leads to a solution, return that solution.
- If an extension does not lead to a solution, backtrack to the previous solution and try a different extension.
- Repeat steps 2-4 until all possible solutions have been explored.

Example: Finding the shortest path through a maze

Algorithm:

1.Start at the starting point.

2.For each of the four possible directions (up, down, left, right), try moving in that direction.

3.If moving in that direction leads to the ending point, return the path taken.

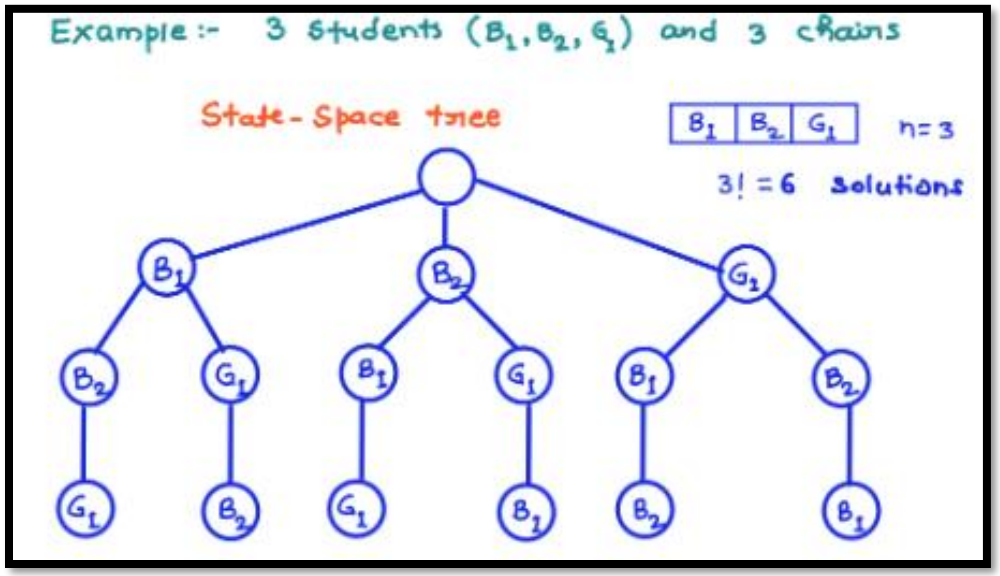
4.If moving in that direction does not lead to the ending point, backtrack to the previous position and try a different direction.

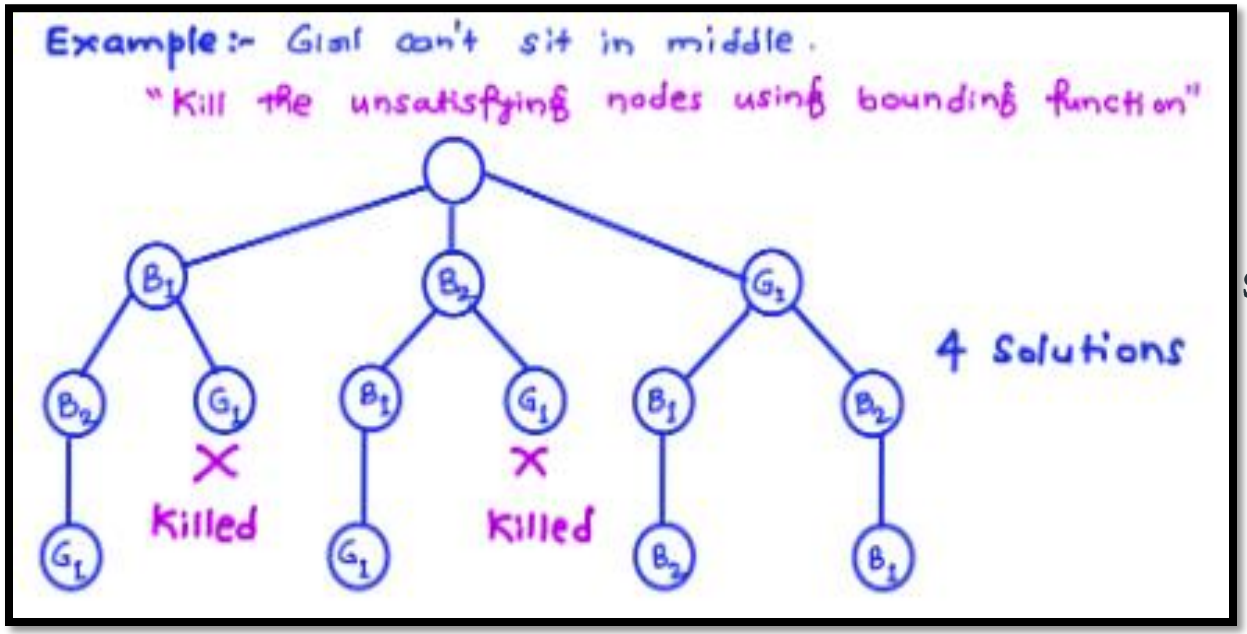
5.Repeat steps 2-4 until the ending point is reached or all possible paths have been explored.

Representation of Backtracking Algorithm

The decision sequences can be represented by a tree – State space tree

A state-space tree is a tree structure that represents all possible sequences of choices or states a problem can take, from an initial state at the root to terminal states (either a complete solution or a dead end) at the leaves.

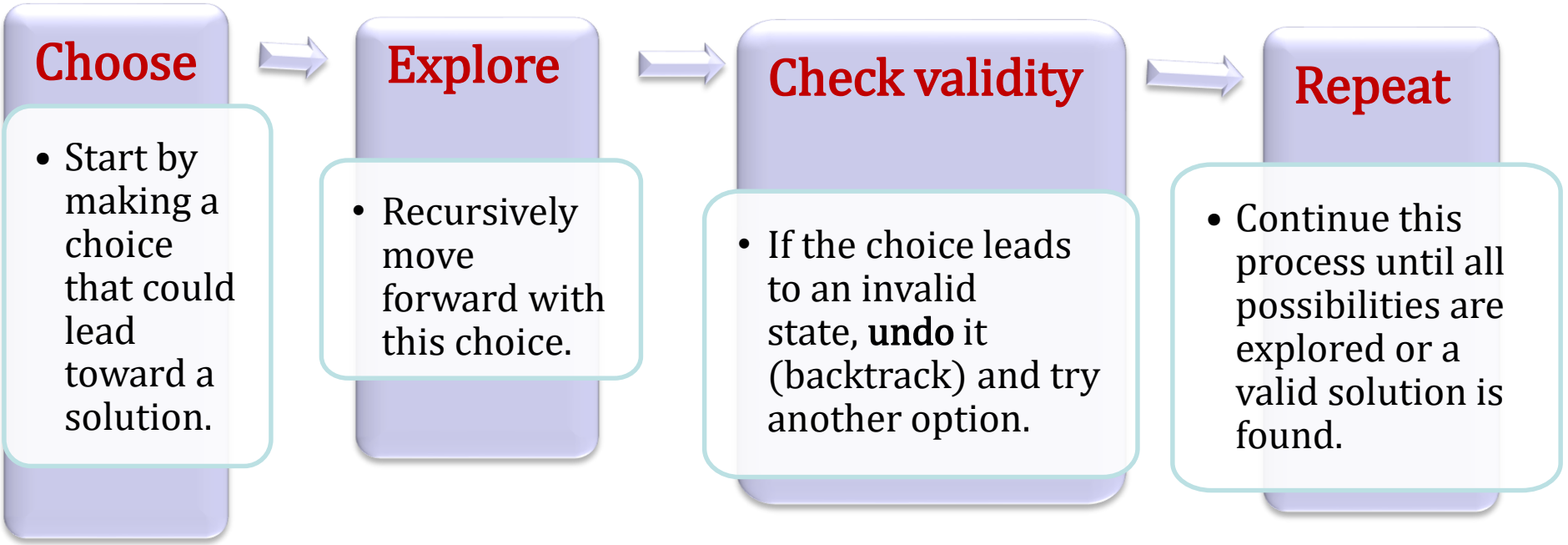




A backtracking algorithm uses the depth-first search method to travel through state space tree, employing a "pruning" strategy to avoid exploring branches that cannot possibly lead to a valid solution.

The Conclusion

The idea is simple:



يضمن التراجع عدم إضاعة الوقت في اتباع مسارات مستحيلة. بدلاً من ذلك، يستكشف بشكل منهجي المسارات الممكنة فقط من خلال التراجع كلما فشل خيار ما.

General outline of Backtracking Algorithm

```
void  try()
{
 initialize selection of candidates;
 do {
      select next candidate;
      if (acceptable)
         {
           record it;
           if (solution incomplete)
              {
                try next step;
                if (not successful)
                     cancel recording;
              }
         }
 } while (not successful)and(remains of
                              candidates);
}
```

Choose step

Explore recursively(DFS)

Backtracking step or Burning step

Repeat step

Examples of Backtracking Algorithm

1) knight's round problem in chess. مسألة جولة حصان الشطرنج
2) Four Queens problem. مسألة الوزراء الأربعة
3) Finding a path through a maze problem. إيجاد طريق خلال المتاهة
4) Target Sum Combinations. مسألة مجموعات المجموع الهدف

Example 1: knight's round problem in chess:

The **Knight's Tour problem** is a classic mathematical puzzle asking if a knight on a chessboard can visit every square exactly once, forming a "Hamiltonian path“

مثال تمهيدي: بفرض لدينا رقعة شطرنج من البعد 3*3 سندرس امكانية تحرك الحصان على هذه الرقعة وتشكيله لمسار هاميلتون.

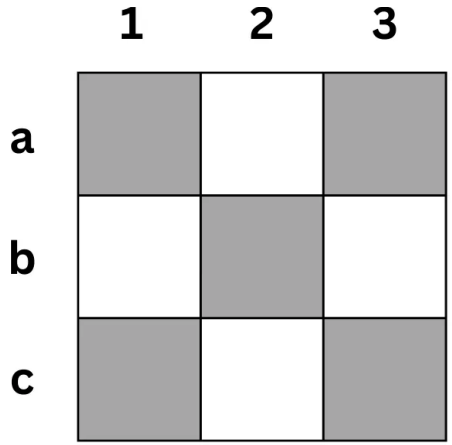
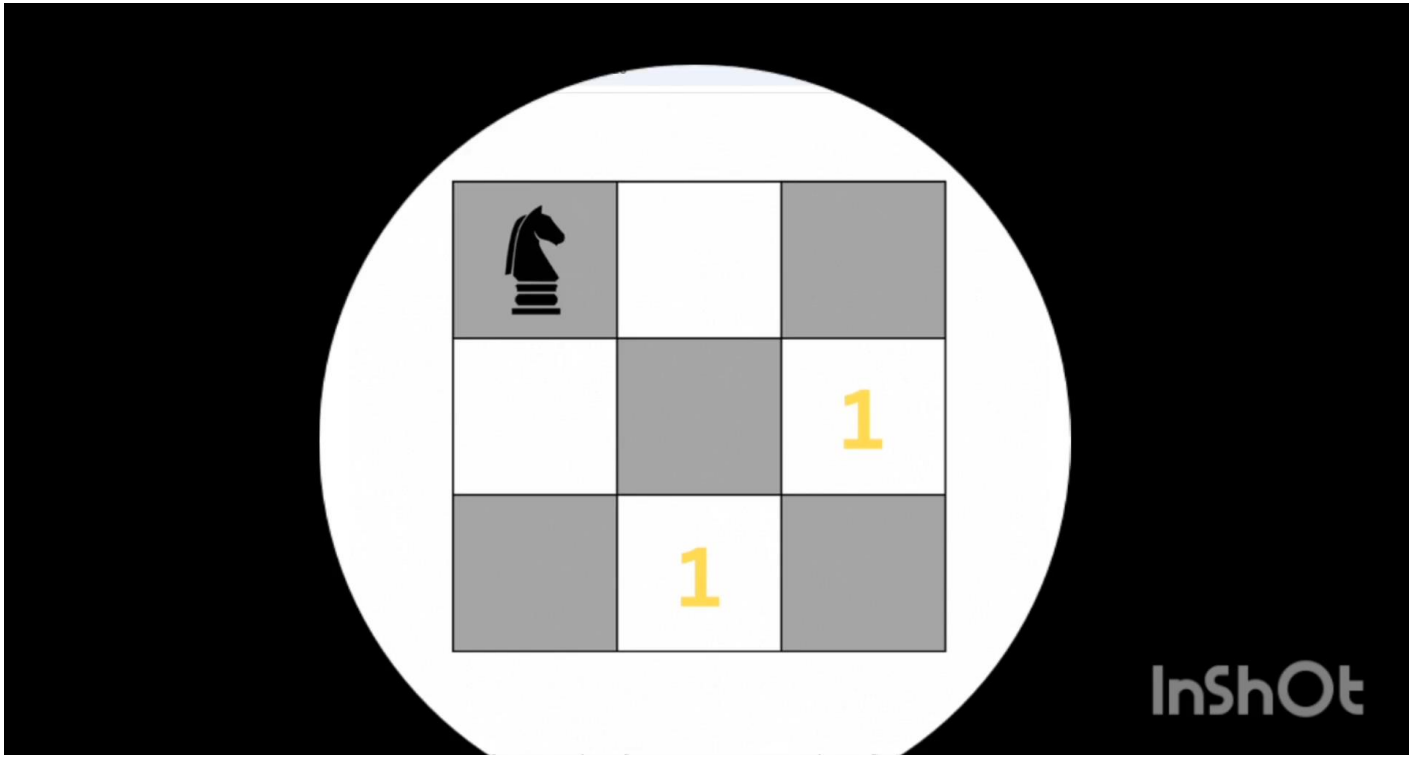


Fig 1. A 3×3 chessboard



لدينا رقعة شطرنج فيها n×n مربعاً، نضع حصاناً في موقع معين. علينا إيجاد طريقة لتغطية الرقعة (إذا كان ذلك ممكناً) أي أن نوجد متتالية من الحركات عددها n^2 – 1 بحيث يحدث المرور بكل مربع مرة واحدة فقط. يمكن التعبير عن ذلك بالخوارزمية العودية المبسطة التالية و المستوحاة من المخطط الخوارزمي التراجعي العام:

```
void  try next move()
{
  initialize selection of moves;
  do {
       select next candidate from list of next moves;
       if (acceptable)
          {
            record move;
            if (board not full)
               { try next move;
                 if (not successful)
                    erase previous recording;
               }
          }
   } while (board not full)and(there are more candidates yet);
}
```

Time Complexity for Backtracking

ندرس التعقيد الزمني للخوارزميات التراجعية عن طريق دراسة كلفة شجرة الحل

1. نبدأ الشجرة بعقدة البداية (الموقع الابتدائي لوقوف الحصان أي أن العقد تشير إلى قرار وهو وضع الحصان في مربع معين في لوحة الشطرنج)
2. المستوى الأول من الشجرة يمثل العدد الأعظمي للمواقع الممكن الانتقال اليها من الموقع الحالي (وهي 8 أماكن - بحسب تطبيق شرط حركة الحصان)
3. المستوى الثاني من اجل كل عقدة سابقة لدينا 8 احتمالات على الاكثر –فعدد العقد على الأكثرفي المستوي الثاني هو 8*8
4. وهكذا حتى الوصول إلى آخر مستوى(عمق الشجرة) والذي رقمه n^2 لان كل مستوي هو رقم حركة.

بالتالي عدد العقد الكلي(أسوء حالة):

هو مجموع العقد في شجرة البحث:

1+8+8^2+8^3+…..+8^n^2

وهي متسلسلة هندسية مجموعها من الشكل :

$$1+q+q^2+q^3+.....+q^n=\frac{q^{n+1}-1}{q-1}$$

$$\frac{8}{7}8^{n^2}-\frac{1}{7}$$

وبالتالي التعقيد الزمني:

$$O(8^{n^2})$$

لتفصيل الخوارزمية:

نمثل رقعة الشطرنج بمصفوفة مربعة h[n][n] حيث n عدد ثابت، ونعتبر أن قيم هذه المصفوفة ستكون على الشكل:

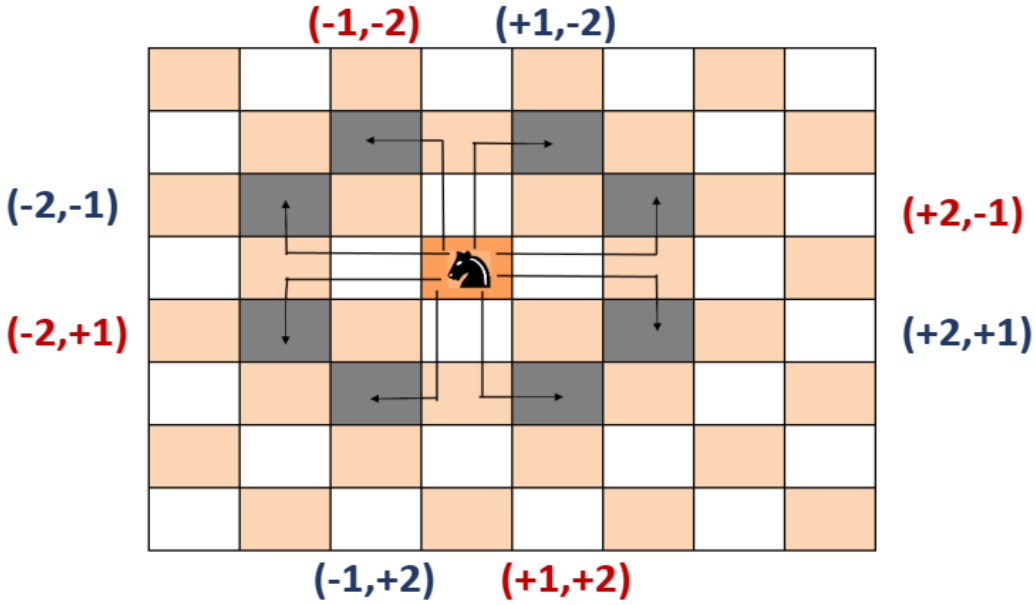
h[u][v] = 0 , When no passage occurs in the square (u,v)

h[u][v] = i , When a pass occurs in the square (u,v) at i^{th} $step$

; $0 \le u < n$ & $0 \le v < n$

كما نعرف مصفوفتين (1-D) للتعبير عن حركة الحصان الممكنة:

```
int yMove[8] = { +2, +1, -1, -2, -2,  -1, +1, +2 };
int xMove[8] = { +1, +2, -2, +1, -1, -2, -2,  -1 };
```



معاملات(وسطاء) الخوارزمية التراجعية(الدالة العودية):

موقع الحصان الحالي u,v (حيث تكون البداية من h[0][0])، رقم الخطوة i (عدد الخطوات الكلي $n^2 - 1$)، ومتحول خرج منطقي q يشير إلى النجاح او الفشل(اكتمال الحل او عدمه)

```
void try1(int i,int x,int y,char &q)
{
  int u,v,k=-1;
  do {
       k++;
       u=x+a[k];  v=y+b[k];
       if ((u>=0)&&(u<n)&&(v>=0)&&(v<n)&&(h[u][v]==0))
          {
            h[u][v]=i;
            if (i<n*n)
               {
                 try1(i+1,u,v,q);
                 if (q=='n')
                    h[u][v]=0;
               }
            else
               q='y';              //  yes , the solution is complete
          }
       } while ((q=='n')&&(k<7));
}
```

Examples:

Input: *n = 5*
Output:
[[0, 5, 14, 9, 20],
[13, 8, 19, 4, 15],
[18, 1, 6, 21, 10],
[7, 12, 23, 16, 3],
[24, 17, 2, 11, 22]]
Explanation: *Each number represents the step at which the Knight visits that cell, starting from (0, 0) as step 0. The output shows one valid Knight's Tour on a 5×5 board.*

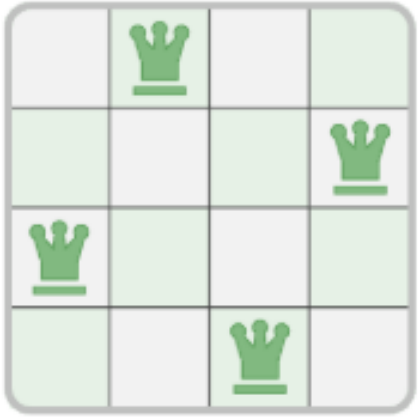
Input: *n = 3*
Output: *[-1]*
Explanation: *It is not possible to find a valid Knight's Tour on a 3x3 chessboard since the Knight cannot visit all 9 cells exactly once without revisiting or getting stuck.*

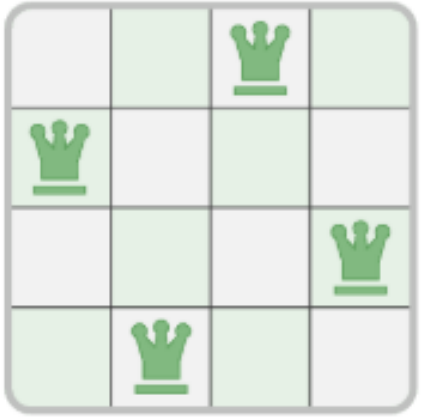
Example 2: Four Queens problem in chess:

The **4-Queens Problem** is a classic puzzle in computer science and mathematics where you must place four queens on a 4x4 chessboard so that no two queens attack each other, meaning no two share the same row, column, or diagonal.

How it's Solved (Backtracking)

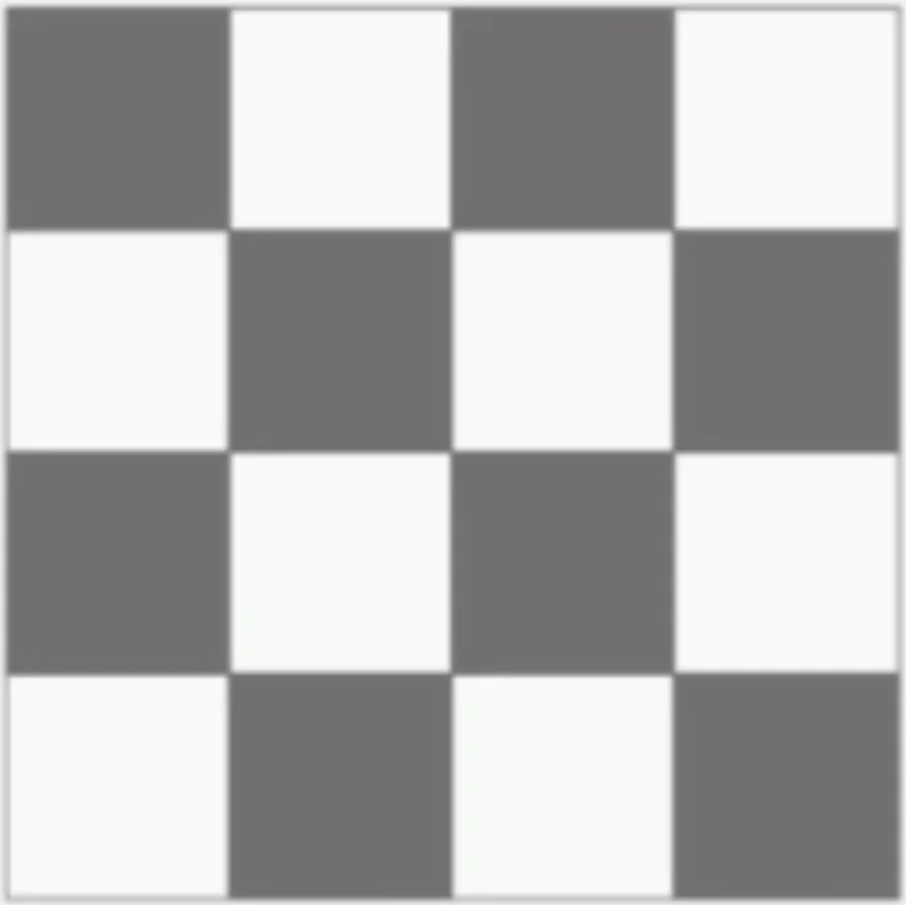
1.Start: Place the first queen in the first row (*or first column*).
2.Explore: Try placing the next queen in the second row/*column*, checking for conflicts (same column/*row* or diagonal).
3.Backtrack: If a placement leads to a conflict (no safe spot in a row/column), go back to the previous queen and try a different position.
4.Repeat: Continue this depth-first search until all four queens are placed safely.





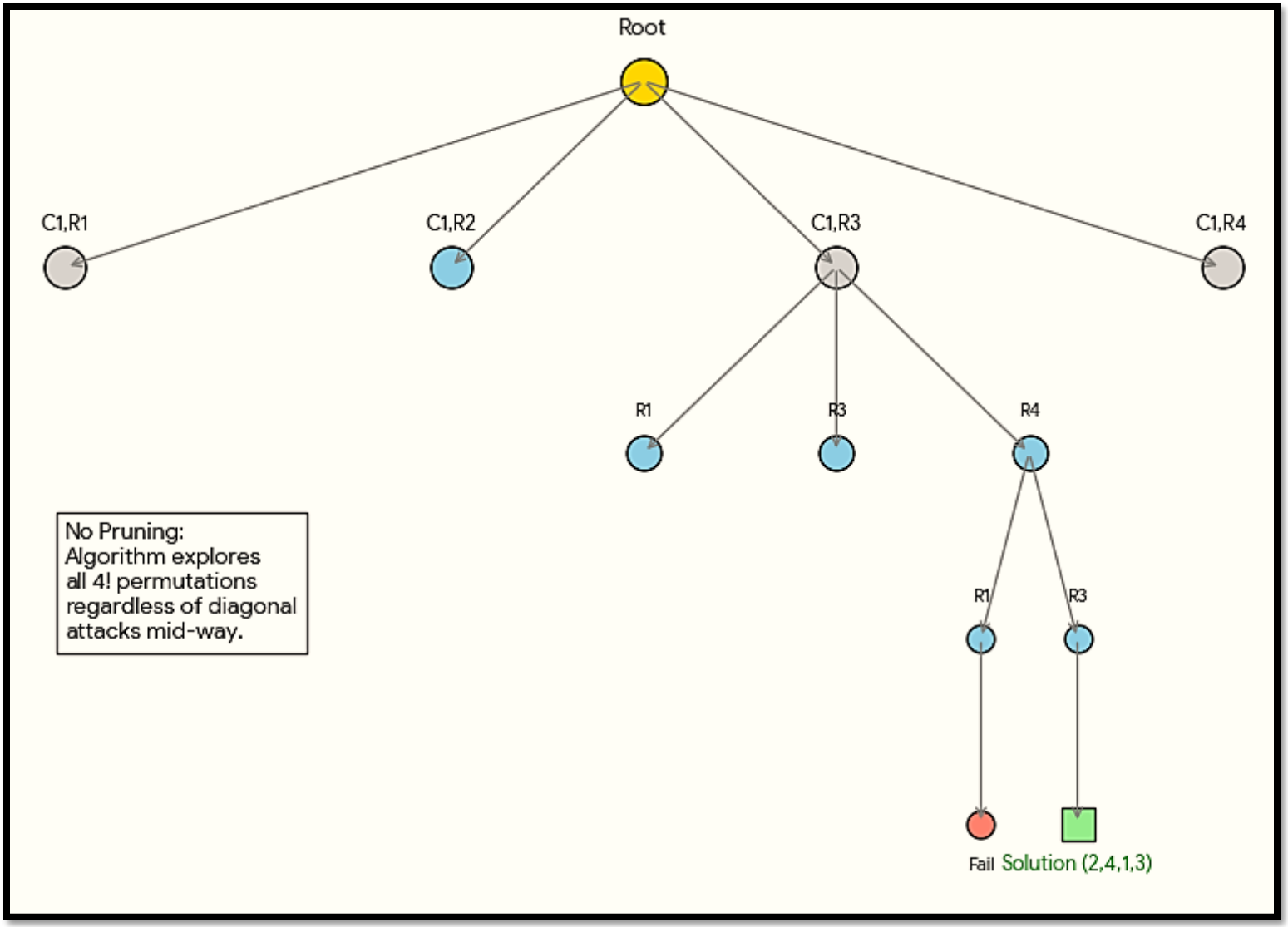
Solution to the 4-queens problem

Step 0: Initialize a 4x4 board.



4 x 4 Chess Board

Part of Space state tree for 4-Queen (No pruning)



The algorithm is smart enough to avoid placing two queens in the same row from the start (Row Constraint)..

Time Complexity

من خلال دراسة كلفة شجرة الحل يمكن أن نستنتج:

Time Complexity: O(n!)

Explanation: In the worst-case scenario, the algorithm explores all possible configurations. For the first row, there are 4 column options; for the second row, there are 3 remaining options, and so on.

مجموع العقد الكلي للشجرة السابقة على الأكثر يمكن أن نعبر عنه بالشكل:

$$S = n + n(n-1) + n(n-1)(n-2) + \ldots + n(n-1)(n-2)\cdots 1$$

The series can be rewritten in reverse order, which helps in identifying a pattern related to the constant **e**:

$$S = n! + \frac{n!}{1!} + \frac{n!}{2!} + \ldots + \frac{n!}{(n-1)!}$$

Factoring out n!, the sum inside the parentheses is:

$$S = n!\left(\frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \ldots + \frac{1}{(n-1)!}\right)$$

But!!

$$e = \sum_{k=0}^{\infty} \frac{1}{k!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \ldots$$

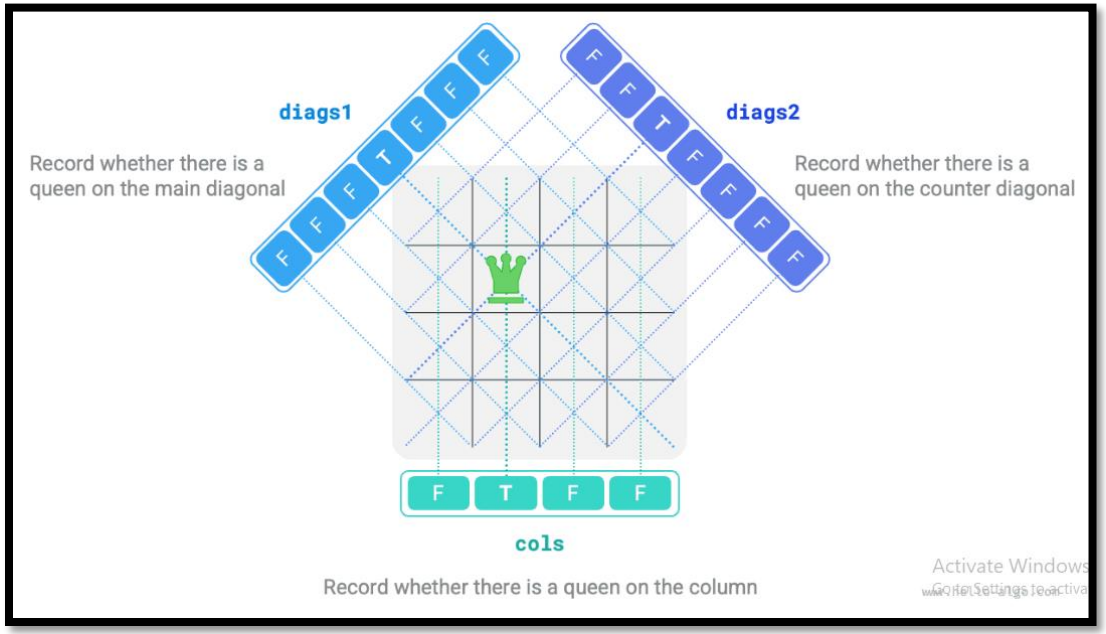
So, $s \cong n! \,.\, e$

Mean the time complexity is O(n!)

Details of algorithm:

اختبار كون الخانة شاغرة أم لا.

- العمود: يكون آمن بما أنّ كل عمود يوجد فيه وزير واحد فقط.
- السطر: نقوم باختباره حيث نستخدم مصفوفة أحادية (بوليانية) حجمها n (عدد الأسطر)، حيث أنّه من أجل الخانة k، تدل القيمة $true$ على أنّ السطر k مهدد (غير آمن)، والقيمة $false$ على أنّ السطر شاغر.
- القطر الرئيسي: نستخدم مصفوفة أحادية (بوليانيّة) حجمها هو عدد الأقطار الرئيسية $(2n-1)$، نلاحظ في الأقطار الرئيسية أنّ عناصر القطر الواحد يمكن ربطها بالعلاقة $(i-j)$، حيث i: دليل السطر، j: دليل العمود، ومن أجل الخانة k، تدل القيمة $true$ على أنّ القطر الرئيسي k مهدد (غير آمن)، وإلّا فإنّ القطر شاغر.
- القطر الثانوي: نستخدم مصفوفة أحادية (بوليانيّة) حجمها هو عدد الأقطار الثانوية $(2n-1)$. نلاحظ في الأقطار الثانويّة أنّ عناصر القطر الواحد يمكن ربطها بالعلاقة $(i+j)$، حيث i: دليل السطر، j: دليل العمود،. ومن أجل الخانة k، تدل القيمة $true$ على أنّ القطر الثانوي k مهدد (غير آمن)، وإلّا فإنّ القطر شاغر.



$a[n]: array\ of\ Booleans$: نخزّن في الخانة i القيمة $true$ في حال كون السطر i مشغولاً.	$X[n]: array\ of\ integers$: نخزّن في الخانة i رقم السطر الموافق للخانة التي وضع فيها الوزير i.
$b[2n-1]: array\ of\ Booleans$: نخزّن في الخانة i القيمة $true$ في حال كون القطر الثانوي i مشغولاً.	$c[n]: array\ of\ Booleans$: نخزّن في الخانة i القيمة $true$ في حال كون القطر الرئيسي رقم i مشغولاً.

الخوارزمية

```
Try(i)
for (j = 1 → n)
if ( a[j] = F and b[i − j] = F and c[i + j] = F)
    x[i] = j , a[j] = T , b[i − j] = T , c[i + j] = T
    if (i < n)
        Try(i + 1)
    else
        print (x)
    x[j] = −1, a[j] = F , b[i − j] = F , c[i + j] = F
```

Homework

حاول حل المسألة التالية وفق مبدأ الخوارزميات التراجعية وادرس تعقيدها:

ليكن لدينا n سؤال امتحاني، وكل سؤال له زمن حل وعلامة ضمن زمن كلي محدود $(Limit\ time)$، والمطلوب إيجاد أفضل مجموعة أسئلة يمكن حلها من أجل الحصول على أعلى علامة.

مثال:

لدينا وفق الجدول التالي (5) أسئلة موضحاً فيه الوقت اللازم لكل سؤال وعلامته، حيث:

	Q_1	Q_2	Q_3	Q_4	Q_5
$Time$	20	10	5	15	5
$Mark$	30	10	10	10	10

زمن الامتحان $Limit\ time = 30\ s$.

ومجموع العلامات الكلي هو (70).